

PumpScript

AI Usage Journal — Part 2 (Entries 5–10)

CSE 341 · Concepts of Programming Languages

Gebze Technical University · 22 May 2026

Student Name	Yusuf Eren Nalbant
Student No	220104004049
AI Tool	Claude Sonnet 4.6 (claude.ai)
Entries	6 entries (Part 2: entries 5–10)
Period	9 May 2026 — 21 May 2026

Journal Entries (Part 2)

Entry #5 of 10 — 9 May 2026, 20:00Phase: Design | Claude Sonnet 4.6

Goal: Decide the type system rules for Part 2 — strong typing, coercion policy, and type equivalence for exercise records.

Prompt:

I need to define the type system for PumpScript (Part 2). Should it be strongly typed? Should `int` be implicitly converted to `float`? For exercise records, should I use name equivalence or structural equivalence? Explain each option and recommend one for a fitness DSL, citing Sebesta.

Response (summary): Claude recommended strong typing (Sebesta §6.12) for reliability, `int` to `float` widening coercion (Sebesta §7.4) because fitness users naturally write whole numbers for weights, and name equivalence (Sebesta §6.14) because two exercise records with identical fields are still semantically different movements and should not be interchangeable.

✓ Accepted

All three recommendations matched my own intuitions. Strong typing was already my preference — a fitness coach writing `weight: "heavy"` is making a mistake, not being creative. The `int`-to-`float` coercion argument was convincing: requiring `80.0` instead of `80` would be an unnecessary burden. Name equivalence felt right immediately — a squat and a bench press are not the same thing even if they happen to have the same numeric values on a given day.

× Rejected / Modified

Claude also suggested allowing float-to-int narrowing in some contexts (e.g., rounding `sets: 4.0` to `4`). I rejected this because it silently loses information and `sets: 4.0` is almost certainly a user mistake, not intentional. Rejecting it explicitly is more reliable.

! Errors Caught

No factual errors. All Sebesta section references were correct.

★ Reflection

This session confirmed that type system decisions are easier to justify when you reason from the domain first and then find the matching Sebesta concept, rather than the other way around. The fitness context made every decision obvious once I thought about what a real coach would write.

Entry #6 of 10 — 12 May 2026, 19:30 [E2 — Mandatory]Phase: Design | Claude Sonnet 4.6

Goal: Ask Claude to explain name equivalence vs structural equivalence for PumpScript's exercise record type, and verify against Sebesta §6.14. (*Mandatory Experiment E2*)

Prompt:

Explain the difference between name equivalence and structural equivalence for PumpScript's exercise record type. Give a concrete example showing what would happen under each rule if I have two exercise records -- `squat` and `bench_press` -- with identical fields. Which does Sebesta §6.14 recommend and why?

Response (key excerpt):

Under structural equivalence, `squat` and `bench_press` would be considered the same type if they have the same fields with the same types. Under name equivalence, they are different types regardless of their fields. Sebesta §6.14 notes that name equivalence is safer for user-defined types because it prevents accidental substitution of semantically different records that happen to share the same structure.

✓ Accepted

The explanation matched Sebesta §6.14 accurately. The concrete example — passing a `squat` record where a `bench_press` is expected — made the risk of structural equivalence clear: the type checker would silently accept a semantically wrong program. Name equivalence prevents this, which is exactly what a fitness DSL needs. I used this reasoning directly in D1 §4.5 and §4.8.

× Rejected / Modified

Claude suggested that structural equivalence could be useful for “generic workout templates” where any exercise with the same field structure could be substituted. I rejected this because PumpScript does not have a template or generic record concept — every exercise block has a specific name and that name carries meaning.

! Errors Caught

Claude's example claimed that under structural equivalence, passing a **squat** record to a function expecting **bench_press** "would only fail at runtime." This is incorrect — in PumpScript, both types are checked statically by the type checker, so neither structural nor name equivalence produces a runtime error. The check happens at compile time in both cases; the difference is only in *what* is accepted. I noted this distinction in my D1 rationale.

*** Reflection**

E2 taught me that AI is reliable for explaining textbook concepts but sometimes conflates static and dynamic type checking. In PumpScript everything is checked statically, so the AI's example about runtime failures was wrong. This is a case where knowing your own language's design prevents you from accepting an incorrect explanation.

Entry #7 of 10 — 15 May 2026, 21:00 Phase: Design | Claude Sonnet 4.6

Goal: Understand operational semantics notation and write rules for the **repeat** and **exercise** constructs for D1 §4.4.

Prompt:

I need to write big-step operational semantics for two PumpScript constructs: **repeat** N times B, and **exercise** blocks. Explain the notation (σ , $\Downarrow$) and write the inference rules. The repeat count N is an `INT_LIT`, not an expression.

Response (summary): Claude explained big-step semantics notation: σ is the program state, $\Downarrow$ means "evaluates to". It wrote a base case (N=0, state unchanged) and a recursive step (execute B once, then repeat N-1 times). For the exercise block it wrote a rule showing all fields evaluated in the same environment, producing output without modifying the environment.

✓ Accepted

The base case and recursive step structure for **repeat** was correct and clean. The key insight — that N is evaluated once before the loop, not re-evaluated each iteration — was correctly captured because I specified in my prompt that N is an `INT_LIT`. This went directly into D1 §4.4 with minor formatting changes.

× Rejected / Modified

Claude initially wrote the exercise rule as modifying the environment (adding field names as variables). I rejected this because in PumpScript, exercise fields are not stored as variables — they are evaluated and printed immediately. The environment should be unchanged after an exercise block. I corrected the rule to show $(env, out) \Downarrow (env, out')$.

! Errors Caught

The environment-modification bug above: Claude assumed exercise fields become variable bindings, which is not part of PumpScript's design. Caught by comparing the rule to my D1 §4.6 scope decisions — field names are not added to the environment.

★ Reflection

Writing formal semantics forces you to be precise about things you previously left vague. Before this session I had not explicitly decided whether exercise fields create variable bindings — the semantics rule forced the decision. The answer (no bindings) is consistent with PumpScript having no assignment statement.

Entry #8 of 10 — 18 May 2026, 20:30Phase: Interpreter | Claude Sonnet 4.6

Goal: Understand the tree-walking interpreter architecture and decide how to implement static scoping using an Environment class.

Prompt:

Explain how a tree-walking interpreter works for PumpScript. Focus on how to implement static scoping using an Environment class. The language has no assignment -- values are only bound at routine call time. How should the Environment parent be set when calling a routine?

Response (summary): Claude explained that each node in the AST has an `execute` or `evaluate` method. For static scoping, when a routine is called, a new `Environment` is created with the *global* environment as parent — not the caller's environment. This ensures that variable lookup follows the source text structure, not the call chain.

✓ Accepted

The key decision — `local_env = Environment(parent=self.global_env)` rather than `parent=caller_env` — was exactly what I needed. This single line is what makes scoping static rather than dynamic in the interpreter. The explanation was clear and directly implementable.

✗ Rejected / Modified

Claude suggested adding a “closure” mechanism to capture the defining environment for each routine. This is unnecessary for PumpScript because routines are always defined at the global level — there are no nested routine definitions. A simple global parent is sufficient and simpler.

! Errors Caught

No factual errors. The environment chain explanation was correct.

★ Reflection

Before this session I was worried about implementing the interpreter because I had never written one before. Understanding that static scoping reduces to one design decision — which environment to use as parent — made the whole interpreter feel approachable. The rest is just `isinstance` checks and recursive calls.

Entry #9 of 10 — 19 May 2026, 21:00 [E3 — Mandatory]Phase: Type Checker | Claude Sonnet 4.6

Goal: Ask Claude to implement the PumpScript type checker, then test it on three inputs where the correct answer is known. (*Mandatory Experiment E3*)

Prompt:

Implement the PumpScript type checker in Python. Rules: strong typing, int-to-float coercion for weight field only, name equivalence for exercise records. Check: (1) exercise field types match expected types, (2) routine call argument types match parameter types, (3) if condition must be bool. Use the AST node classes from `ast_nodes.py`.

Test inputs and results:

Test 1 — Valid program:

weight: 80 (int, coercion to float) → [OK] Type check passed.

Test 2 — Wrong field type:

weight: "heavy" → [Type Error] Line 6: field 'weight' expects float but got string

Test 3 — Wrong argument type:

chest_day("four") → [Type Error] Line 11: parameter 'sets' expects int but got string

✓ Accepted

All three test results were correct. The overall architecture was clean: one method per check category (field types, argument types, if condition), a separate `get_type` method for expression type inference, and a `is_compatible` function for the coercion rule. I kept this structure entirely.

× Rejected / Modified

Claude applied the int-to-float coercion to *all* float fields, not just `weight`. In my design, coercion is allowed for any float field (sets expects int, so 4.0 is still rejected). The generated `EXERCISE_FIELD_TYPES` dictionary correctly defines expected types per field, and `is_compatible` handles coercion. No change needed here — the logic was already correct.

! Errors Caught

One subtle bug: Claude's initial `check_if_stmt` accepted any non-zero integer as a truthy condition (Python's `if value:` semantics). PumpScript requires the condition to be explicitly `bool` — `if 42 { }` should be a type error. I fixed this by checking `cond_type != "bool"` explicitly rather than relying on Python's truthiness.

★ Reflection

E3 confirmed the pattern from E1: AI generates correct structure but embeds Python-specific assumptions that don't match the designed language semantics. The if-condition bug is a perfect example — Python allows any truthy value in an if, but PumpScript is strongly typed and requires bool. Knowing your own language's rules is essential for catching these subtle errors.

Entry #10 of 10 — 21 May 2026, 20:00Phase: Writing | Claude Sonnet 4.6

Goal: Write the Design Rationale section (D1 §4.8) for the five major type system and expression decisions made in Part 2.

Prompt:

Write the design rationale for PumpScript's five decisions: (1) strong typing, (2) int-to-float coercion, (3) name equivalence, (4) no boolean connectives, (5) no assignment statement. For each, state the trade-off accepted using Sebesta's evaluation criteria.

Response (summary): Claude produced a rationale paragraph for each decision, citing Sebesta reliability (§1.3.3) and writability (§1.3.2) criteria. Each paragraph named a concrete trade-off: reliability vs writability for coercion, reliability vs flexibility for name equivalence, simplicity vs expressiveness for no boolean connectives, reliability vs generality for no assignment.

✓ Accepted

The trade-off framing was useful and went directly into D1 §4.8. Each decision was tied to a Sebesta criterion, which is exactly what the rubric requires. The structure (one paragraph per decision, one named trade-off each) was clean and defensible in the exam.

✗ Rejected / Modified

Claude's rationale for omitting boolean connectives was too generic — it said "boolean logic is complex" without connecting it to the fitness domain. I rewrote this paragraph to say specifically that fitness programs rarely need compound conditions and that omitting connectives keeps the grammar smaller without reducing real-world expressiveness for the target user.

! Errors Caught

No factual errors. The Sebesta citations were accurate.

★ Reflection

AI-generated rationale tends to be domain-agnostic — it explains why strong typing is good in general, not why it is the right choice for a fitness DSL specifically. The most important editing I did was replacing generic software-engineering arguments with fitness-domain arguments. The exam will ask about my language, not about type systems in general.